

Lightning lesson, sixty minutes

Design a Software Factory for Your Coding Agents

Architecture session. No live coding. The room leaves with a map they can hold against their own repository, and one named gap.

Twenty minutes ahead, alone

Before anyone joins

Goal The demo is warm and the console has data in it. Nothing here happens while people watch.

Say Nothing. This segment is setup.

Run

```
$ make check
$ make score
$ make demo STEP=06
```

Check

- The score prints and every phase is green.
- The console is serving and lists finished runs with their timings.
- Your editor is on a clean branch, because the harness will create worktrees and you want that visible.

Ask *Nobody. Confirm the room can read a terminal at the back before you start.*

0:00 to 0:08

The afternoon that disappears

Goal Everyone recognises the problem as theirs before any architecture appears.

Say

- Open with the shape of the day, not the technology. The function took seconds. Then came re-explaining the codebase, checking what else moved, and tidying the pull request.
- Name the wrong conclusion out loud: that the model needs to improve. Ask who has responded to a bad result by adding another paragraph to their instructions.
- Land the reframe. The generator is not the missing piece. There is nowhere safe for it to run, no reliable way to give it what it needs, and nothing checking its work before it reaches you.

Run Nothing yet. Resist opening a terminal here.

Room sees Slides or a whiteboard only. Faces, not a screen.

Ask *How long is your instructions file now, and when did you last read all of it?*

0:08 to 0:28

The map

Goal Attendees can name all six jobs and say which failure each one removes.

Say

- Walk the six in order, giving each one a single sentence and a symptom. Do not define them abstractly; define them by what goes wrong when they are missing.
- Spend the longest on the second and fifth, because those are where the room's pain actually is.
- Say the sentence that reframes the rest of the hour: these fail separately, so the fix differs depending on which one is absent.

Run Show the map in the concept spine and scroll through it.

Room sees

- The six-layer diagram, one heading at a time.
- The symptom under each, said aloud rather than written on the slide.

Ask *Pick the symptom you had this week. Which job was missing?*

0:28 to 0:40

Routing beats piling

Goal The room understands why their instructions file stopped helping, and what replaces it.

- Say**
- Describe the growth curve honestly. Each addition works, which is why it keeps happening, and accuracy still falls.
 - Explain the competition for attention: the sentence that matters is now one of forty, thirty-nine of which are irrelevant to this task.
 - Show the alternative as a calculation rather than a philosophy. Files to be touched, owners of those files, rules for those owners, skills for that kind of work.
-

Run

```
$ make demo STEP=03
```

- Room sees**
- Two tasks, side by side, receiving visibly different material.
 - The frontend task never being shown the money rules, and the reason why printed next to it.
-

Ask *What does a new engineer get told on day one that your agent is never told?*

0:40 to 0:52

The whole loop, once

Goal They see the finished thing move, so the layers stop being a diagram.

- Say**
- Narrate the stages as they happen and keep naming which job is doing the work.
 - Stop on the refusal and give it a beat. A run declining to touch the money path is the most important thing on the screen, and it looks like nothing.
 - Close the loop at the pull request and say plainly that this is where the machine stops.
-

Run

```
$ make demo STEP=06
```

- Room sees**
- An item claimed, a worktree created, checks running, a verdict, a draft pull request.
 - A second item stopping with a policy reason instead of a change.
 - The console updating with the stage timings for the run that just finished.
-

Ask *Which of those stages would you have let run unattended on your repository this morning?*

0:52 to 1:00

One gap each

Goal Nobody leaves with a reading list. Everybody leaves with one named absence.

Say

- Ask them to write down which of the six they do not have, in their own words, before the call ends.
- Say where the next sessions go and what each one builds, briefly, without selling.

Run Leave the console on screen while taking questions.

Room sees The repository address, and the concept spine open at the overview.

Ask *Name the one you are missing. Type it in the chat so the room sees the spread.*

0:35 to 0:45

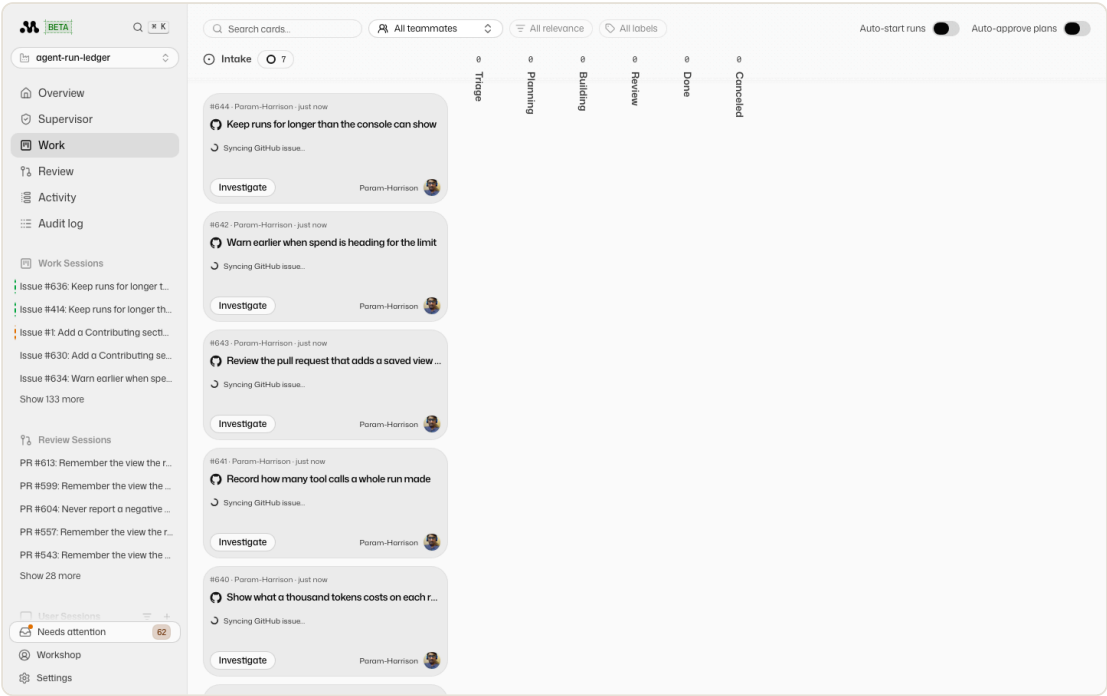
The same policy, read by a product

Goal They have seen the charter they just wrote being obeyed by software none of us built.

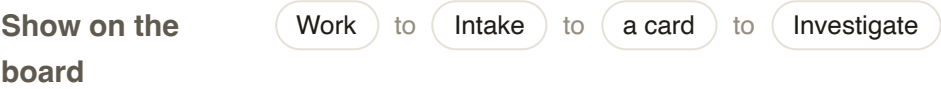
- Say**
- Point out that the board is reading the same `.factory`` directory the hand-built loop reads.
 - The value of a written policy is that a second implementation can honour it without being told anything else.
 - Name what is missing on purpose: nothing has run yet, so nothing on screen is a claim about quality.

Run

```
$ make factory-up
$ make lab-reset
```



Every issue the repository declares arrives from one place. Nothing has started, and both switches are off.



The charter is not read by magic

Mastra Factory has no knowledge of a .factory directory. What it reads is AGENTS.md, and the only reason these rules bind anything is that the file points at them and repeats the protected list. Delete it and every limit here becomes a comment.

Both switches are off, deliberately

Arrival and action are separate. An issue landing on Intake is not a run starting, and the gap between those two is where a person decides whether this is work at all.

Six cards, one repository

Nothing on this board knows what the ledger is. It knows a repository was connected and that its issues arrive here.

Room does

- Watches six cards appear without anybody filing them.
- Notices that Auto-start runs is off, so arrival and action are separate things.

Ask

If your policy file vanished, which of these six would a stranger get wrong first?

Concepts for this session: six layers, boundary, context, skills, execution, verification, delivery, the growing prompt.